

Y طيب انا جيتت الكاميرا اهه خلينا نجربها يلا على ubuntu 22 on ros 2 humble
o

ألف مبروك! اختيار ممتاز وميوافر عليك وقت ومجهود كبير جداً. بما إنك اشتريت **Orbbec Astra Pro**، أنت
دلوقتي جاهز تشتغل على **Ubuntu 22.04** و **ROS 2 Humble** بشكل مباشر (Native) وبدون أي
a
i
d
تعقيدات.

الكاميرا دي بتنقسم لجزئين داخلياً:

1. **Depth/IR**: بيشتغل ببروتوكول OpenNI (وده المسؤول عنه تعريف Orbbec).

2. **Color (RGB)**: بتشتغل ببروتوكول UVC (زي أي ويب كام عادية).

عشان نشغلها على ROS 2 Humble، هنستخدم حزمة ros2_astra_camera الرسمية.
اتبع الخطوات دي بالترتيب:

1. تجهيز النظام (System Dependencies)

افتح التيرمينال (Ctrl+Alt+T) ونزل المكتبات الضرورية عشان الكاميرا تشتغل:

Bash

```
sudo apt update
```

```
\ sudo apt install libgflags-dev nlohmann-json3-dev libgoogle-glog-dev
```

```
ros-humble-image-transport ros-humble-image-publisher
```

2. تجهيز مساحة عمل ROS 2 (Workspace)

لو معندكش ros2_ws جاهز، منعمل واحد جديد:

Bash

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws/src
```

3. تحميل تعريف الكاميرا (Cloning the Driver)

هنا نزل الحزمة الرسمية من Orbbec المخصصة لـ ROS 2:

Bash

```
git clone https://github.com/orbbec/ros2_astra_camera.git
```

4. تثبيت قواعد الـ (USB Udev Rules)

الخطوة دي مهمة جداً عشان اللينكس يشوف الكاميرا من غير sudo:

Bash

```
cd ~/ros2_ws/src/ros2_astra_camera/astra_camera/scripts
```

```
sudo bash install.sh
```

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

(لو طلب منك ريستارت، يفضل تعمل ريستارت، بس غالباً الأمر الأخير بيكفي).

5. البناء (Build)

نرجع للمجلد الرئيسي ونبني الحزمة:

Bash

```
cd ~/ros2_ws
```

```
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release
```

(العملية دي ممكن تاخذ دقيقتين).

6. التشغيل والتجربة (Launch)

بعد ما البناء يخلص، وصل الكاميرا في الـ USB، واكتب الأوامر دي:

في التيرمينال الأول (تشغيل الكاميرا):

Bash

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 launch astra_camera astra_mini.launch.xml
```

(ملحوظة: ملف `astra_mini.launch.xml` غالباً يشغل الـ *Astra Pro* برضه لأنهم نفس المعمارية، لو ملقيتش ملف باسم `astra_pro` جرب ده).

في تيرمينال ثاني (عشان نشوف الصورة):

Bash

```
ros2 run rviz2 rviz2
```

جوه **Rviz**:

1. اضغط **Add**.

2. اختر تبويب **By Topic**.

3. دور على `camera/color/image_raw/` واختار **Image**.

4. دور على `camera/depth/image_raw/` واختار **Image** أو **PointCloud2**.

5. في خانة **Fixed Frame** (على الشمال فوق)، غيرها خليها `camera_link` أو `openni_depth_optical_frame`.

□ تلميح مهم للـ **Astra Pro**: بما إن الـ *Astra Pro* كاميرا الألوان فيها (RGB) منفصلة (UVC)، ساعات بتحتاج نشغلها كـ "ويب كام" عادية لو التعريف ملقطهاش أوتوماتيك. لو شوفت العمق (Depth) بس والألوان لا، قولي وهنزل `usb_cam` باكدج بسيطة تشغل الألوان لوحدها.

جرب الخطوات دي وقولي النتيجة!

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 launch astra_camera astra_mini.launch.xml
```

not found:

```
"/home/abdullah/ros2_ws/install/astra_camera_msgs/share/astra_camera_msgs/local_setup.bash"
```

```
Package 'astra_camera' not found: "package 'astra_camera' not found,
searching: ['/home/abdullah/ros2_ws/install/wheel_teleop',
'/home/abdullah/ros2_ws/install/hub_motor_controller',
```

```
'/home/abdullah/ros2_ws/install/encoder_odometry',  
'/home/abdullah/ros2_ws/install/diff_drive_bridge', '/opt/ros/humble']"  
abdullah@abdullah-sherif:~/ros2_ws$
```

المشكلة هنا واضحة: الحزمة (**Package**) لم يتم بناؤها (**Build**) بشكل صحيح، أو أن هناك ملفات ناقصة، لذلك نظام ROS 2 لا يراها حتى بعد كتابة أمر `source`.

الخطأ Package 'astra_camera' not found يعني أن أمر `colcon build` لم ينجح في إنشاء مجلدات الحزمة، غالباً بسبب نقص في المكتبات (**Dependencies**) قبل عملية البناء.

لحل المشكلة نهائياً، اتبع هذه الخطوات بالترتيب (خطوات "تنظيف وإعادة بناء"):

1. تثبيت الاعتمادات الناقصة (خطوة مهمة جداً)

أغلب مشاكل البناء تحدث بسبب نسيان هذه الخطوة. سنستخدم أداة `rosdep` لتنزيل أي مكتبة ناقصة للكاميرا أوتوماتيكياً.

افتح التيرمينال واكتب:

Bash

```
cd ~/ros2_ws  
sudo apt update  
rosdep update  
rosdep install --from-paths src --ignore-src -r -y  
(هذا الأمر سيفحص كود الكاميرا وينزل أي مكتبة ناقصة مثل glog أو magic_enum وغيرها).
```

2. التنظيف وإعادة البناء (Clean & Rebuild)

سنقوم بمسح محاولات البناء القديمة الفاشلة ونبني من جديد لنرى أي خطأ بوضوح.

Bash

```
cd ~/ros2_ws  
/rm -rf build/ install/ log  
colcon build --symlink-install --packages-select astra_camera  
astra_camera_msgs
```

انتظر حتى ينتهي هذا الأمر.

● إذا ظهرت كلمة "**Summary: 2 packages finished**" باللون الأخضر، فالبناء نجح.

● إذا ظهرت كلمة "**Failed**" باللون الأحمر، انسخ لي رسالة الخطأ (غالباً ستكون بسبب مكتبة ناقصة).

3. التشغيل والتشغيل (Source & Launch)

بمجرد نجاح الخطوة السابقة، نفذ الآتي:

Bash

```
source install/setup.bash
```

```
ros2 launch astra_camera astra_mini.launch.xml
```

□ **ملاحظة:** إذا كنت تستخدم **Astra Pro** (وليس Astra العادية)، قد تحتاج لتشغيل ملف إطلاق مختلف إذا لم يعمل `astra_mini`. جرب هذا الأمر بدلاً منه إذا لزم الأمر:

Bash

```
ros2 launch astra_camera astra_pro.launch.xml
```

(لكن جرب `astra_mini` أولاً لأنه الأغلب توافقاً مع التعريف الجديد).

جرب وبلغني إذا ظهرت أي رسالة خطأ أثناء `colcon build`.

```
abdullah@abdullah-sherif:~/ros2_ws$ cd ~/ros2_ws
```

```
rm -rf build/ install/ log/
```

```
colcon build --symlink-install --packages-select astra_camera
```

```
astra_camera_msgs
```

```
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
```

```
'/home/abdullah/ros2_ws/install/wheel_teleop' in the environment variable
```

```
AMENT_PREFIX_PATH doesn't exist
```

```
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment
variable AMENT_PREFIX_PATH doesn't exist
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment variable
AMENT_PREFIX_PATH doesn't exist
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
'/home/abdullah/ros2_ws/install/diff_drive_bridge' in the environment variable
AMENT_PREFIX_PATH doesn't exist
[0.264s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment
variable CMAKE_PREFIX_PATH doesn't exist
[0.264s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment variable
CMAKE_PREFIX_PATH doesn't exist
Starting >>> astra_camera_msgs
--- stderr: astra_camera_msgs
/usr/bin/ld: /usr/local/lib/libpython3.10.a(ast_opt.o): warning: relocation against
`PyExc_RecursionError' in read-only section `.text.unlikely'
/usr/bin/ld: /usr/local/lib/libpython3.10.a(boolobject.o): relocation
R_X86_64_PC32 against symbol `_Py_TrueStruct' can not be used when making
a shared object; recompile with -fPIC
/usr/bin/ld: final link failed: bad value
collect2: error: ld returned 1 exit status
gmake[2]: ***
[CMakeFiles/astra_camera_msgs__rosidl_generator_py.dir/build.make:243:
rosidl_generator_py/astra_camera_msgs/libastra_camera_msgs__rosidl_generat
or_py.so] Error 1
gmake[1]: *** [CMakeFiles/Makefile2:475:
CMakeFiles/astra_camera_msgs__rosidl_generator_py.dir/all] Error 2
gmake: *** [Makefile:146: all] Error 2
---
Failed <<< astra_camera_msgs [6.38s, exited with code 2]
```

Summary: 0 packages finished [6.62s]

1 package failed: astra_camera_msgs

1 package had stderr output: astra_camera_msgs

1 package not processed

usr/bin/lib/libpython3.10.a ... recompile with -fPIC/: رسالة الخطأ:

السبب هو أن لديك نسخة **Python** مثبتة يدوياً في المسار `usr/local/lib/` (غالباً قمت بتثبيتها بنفسك أو جاءت مع برنامج آخر)، وهي تتعارض مع نسخة الـ `Python` الأصلية الخاصة بالنظام (`Ubuntu System Python`). نظام `ROS 2` يحاول استخدام النسخة اليدوية هذه لكنها "مكتبة ثابتة" (`Static Library`) لا تصلح لبناء البرامج المشتركة، ولذلك يفشل الرابط (`Linker`).

الحل هو إجبار `colcon` على استخدام بايثون النظام الأصلي وتجاهل النسخة اليدوية.

نفذ الأمر التالي بدقة (سيقوم بمسح القديم وإعادة البناء مع تحديد مسار البايثون الصحيح):

Bash

1. تنظيف مخلفات البناء السابقة

```
/rm -rf build/ install/ log
```

2. إعادة البناء مع إجبار النظام على استخدام مكتبة بايثون الصحيحة

```
\ colcon build --symlink-install
```

```
\ packages-select astra_camera astra_camera_msgs - -
```

```
\ cmake-args - -
```

```
\ DPython3_EXECUTABLE=/usr/bin/python3 -
```

```
\ DPython3_INCLUDE_DIR=/usr/include/python3.10 -
```

```
DPython3_LIBRARY=/usr/lib/x86_64-linux-gnu/libpython3.10.so -
```

ملاحظة: تجاهل التحذيرات الصفراء في البداية (`WARNING: ... path doesn't exist`)، هذا طبيعي لأننا حذفنا مجلد `install` يدوياً. المهم هو أن ينتهي الأمر بكلمة **Summary: 2 packages finished**.

بمجرد النجاح، لا تنس تفعيل الحزمة:

Bash

```
source install/setup.bash
```

ثم جرب التشغيل.

1. تنظيف مخلفات البناء السابقة

```
rm -rf build/ install/ log/
```

2. إعادة البناء مع إجبار النظام على استخدام مكتبة بايثون الصحيحة

```
colcon build --symlink-install \
```

```
--packages-select astra_camera astra_camera_msgs \
```

```
--cmake-args \
```

```
-DPython3_EXECUTABLE=/usr/bin/python3 \
```

```
-DPython3_INCLUDE_DIR=/usr/include/python3.10 \
```

```
-DPython3_LIBRARY=/usr/lib/x86_64-linux-gnu/libpython3.10.so
```

```
[0.262s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
```

```
'/home/abdullah/ros2_ws/install/wheel_teleop' in the environment variable  
AMENT_PREFIX_PATH doesn't exist
```

```
[0.262s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
```

```
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment  
variable AMENT_PREFIX_PATH doesn't exist
```

```
[0.262s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
```

```
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment variable  
AMENT_PREFIX_PATH doesn't exist
```

```
[0.262s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
```

```
'/home/abdullah/ros2_ws/install/diff_drive_bridge' in the environment  
variable AMENT_PREFIX_PATH doesn't exist
```

```
[0.262s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path
```

```
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment  
variable CMAKE_PREFIX_PATH doesn't exist
```

```
[0.262s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path
```

```
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment  
variable CMAKE_PREFIX_PATH doesn't exist
```

```
Starting >>> astra_camera_msgs
```

```
--- stderr: astra_camera_msgs
```

```
CMake Warning:
```

```
Manually-specified variables were not used by the project:
```

```
Python3_INCLUDE_DIR
Python3_LIBRARY
```

```
/usr/bin/ld: /usr/local/lib/libpython3.10.a(ast_opt.o): warning: relocation against
`PyExc_RecursionError' in read-only section `.text.unlikely'
/usr/bin/ld: /usr/local/lib/libpython3.10.a(boolobject.o): relocation
R_X86_64_PC32 against symbol `_Py_TrueStruct' can not be used when making
a shared object; recompile with -fPIC
/usr/bin/ld: final link failed: bad value
collect2: error: ld returned 1 exit status
gmake[2]: ***
[CMakeFiles/astra_camera_msgs__rosidl_generator_py.dir/build.make:243:
rosidl_generator_py/astra_camera_msgs/libastra_camera_msgs__rosidl_generat
or_py.so] Error 1
gmake[1]: *** [CMakeFiles/Makefile2:475:
CMakeFiles/astra_camera_msgs__rosidl_generator_py.dir/all] Error 2
gmake: *** [Makefile:146: all] Error 2
---
Failed <<< astra_camera_msgs [6.35s, exited with code 2]
```

```
Summary: 0 packages finished [6.59s]
 1 package failed: astra_camera_msgs
 1 package had stderr output: astra_camera_msgs
 1 package not processed
abdullah@abdullah-sherif:~/ros2_ws$
```

المشكلة واضحة جداً الآن، والسبب هو ملف "خبث" موجود عندك في النظام يفسد عملية البناء مهماً غيرنا في الإعدادات.

تحليل الخطأ: الرابط (Linker) يحاول يستخدم ملف اسمه /usr/local/lib/libpython3.10.a. الملف ده عبارة عن نسخة بايثون قديمة أو مثبتة يدوياً (Custom Build) ومش معمولة عشان تشتغل مع البرامج المشتركة (Shared Objects)، وده اللي بيطلع خطأ recompile with -fPIC.

الحل الجفري هو إننا "نخفي" الملف ده مؤقتاً عشان نجبر النظام يستخدم نسخة بايثون الأصلية السليمة اللي جايه مع أوبونتو.

نفذ الخطوات دي بالترتيب ومنتحل فرأ:

1. إخفاء الملف المسبب للمشكلة

افتح التيرمينال واكتب الأمر ده عشان تغير اسم الملف الخاطئ (كأننا بنعمله backup):

Bash

```
sudo mv /usr/local/lib/libpython3.10.a /usr/local/lib/libpython3.10.a.bak
```

(لو طلعلك إن الملف مش موجود، قولي، بس هو موجود حسب رسالة الخطأ).

2. التنظيف وإعادة البناء (بدون تعقيدات)

دلوقتي بما إن الملف "الشريـر" اختفى، النظام هيروح للملف السليم لوحده. ارجع لمجلد الـ workspace ونفذ البناء العادي:

Bash

```
cd ~/ros2_ws
```

```
rm -rf build/ install/ log
```

```
colcon build --symlink-install
```

النتيجة المتوقعة: المرة دي مش هيلاقى /usr/local/lib/libpython3.10.a فهيروح أوتوماتيك لـ /usr/lib/x86_64-linux-gnu/libpython3.10.so (ده الملف السليم)، والبناء هينجح وتظهر لك رسالة **Summary: 2 packages finished**.

جرب وقولي النتيجة!

```
cd ~/ros2_ws
```

```
rm -rf build/ install/ log/
```

```
colcon build --symlink-install
```

```
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
'/home/abdullah/ros2_ws/install/wheel_teleop' in the environment variable
AMENT_PREFIX_PATH doesn't exist
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment
variable AMENT_PREFIX_PATH doesn't exist
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment variable
AMENT_PREFIX_PATH doesn't exist
[0.263s] WARNING:colcon.colcon_ros.prefix_path.ament:The path
'/home/abdullah/ros2_ws/install/diff_drive_bridge' in the environment variable
AMENT_PREFIX_PATH doesn't exist
[0.263s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment
variable CMAKE_PREFIX_PATH doesn't exist
[0.263s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment variable
CMAKE_PREFIX_PATH doesn't exist
Starting >>> astra_camera_msgs
Starting >>> diff_drive_bridge
Starting >>> encoder_odometry
Starting >>> hub_motor_controller
Starting >>> wheel_teleop
Finished <<< diff_drive_bridge [1.35s]
Finished <<< wheel_teleop [1.36s]
Finished <<< encoder_odometry [7.99s]
```

```
Finished <<< astra_camera_msgs [9.50s]
Starting >>> astra_camera
Finished <<< hub_motor_controller [10.6s]
--- stderr: astra_camera
CMake Error at /usr/share/cmake-3.22/Modules/FindPkgConfig.cmake:890
(message):
  None of the required 'libuvc' found
Call Stack (most recent call first):
  CMakeLists.txt:44 (pkg_search_module)
```

```
CMake Error at CMakeLists.txt:46 (message):
  libuvc is not found
```

```
Failed <<< astra_camera [1.77s, exited with code 1]
```

```
Summary: 5 packages finished [11.5s]
 1 package failed: astra_camera
 1 package had stderr output: astra_camera
abdullah@abdullah-sherif:~/ros2_ws$
```

ممتاز! إحنا حلينا مشكلة البايثون، وastra_camera_msgs اتبنت بنجاح (وده كان العائق الأكبر).

المشكلة الجديدة بسيطة جداً: مكتبة libuvc ناقصة. الخطأ None of the required 'libuvc' found معناه إن الكومبايلر بيدور على مكتبة التعامل مع الـ USB Video Class ومش لاقبها.

الحل في خطوة واحدة (تنصيب المكتبة الناقصة):

1. افتح التيرمينال واكتب الأمر ده:

Bash

```
sudo apt update
```

```
sudo apt install libuvc-dev
```

2. بعد ما يخلص تثبيت، ارجع ابن الحزمة تاني:

Bash

```
cd ~/ros2_ws
```

```
colcon build --symlink-install --packages-select astra_camera
```

(ملحوظة: أنا حددت astra_camera بس في أمر البناء عشان نوفر وقت، لأن باقي الحزم خلصت خلاص).

بمجرد ما تخلص، جرب تشغل الكاميرا:

Bash

```
source install/setup.bash
```

```
ros2 launch astra_camera astra_mini.launch.xml
```

(أو `astra_pro.launch.xml` لو الأولاني مشغلش).

```
cd ~/ros2_ws
```

```
colcon build --symlink-install --packages-select astra_camera
```

```
[0.264s] WARNING:colcon.colcon_ros.prefix_path.ament:The path  
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment  
variable AMENT_PREFIX_PATH doesn't exist
```

```
[0.264s] WARNING:colcon.colcon_ros.prefix_path.ament:The path  
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment variable  
AMENT_PREFIX_PATH doesn't exist
```

```
[0.264s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path  
'/home/abdullah/ros2_ws/install/hub_motor_controller' in the environment  
variable CMAKE_PREFIX_PATH doesn't exist
```

```
[0.264s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path  
'/home/abdullah/ros2_ws/install/encoder_odometry' in the environment variable  
CMAKE_PREFIX_PATH doesn't exist
```

```
Starting >>> astra_camera
```

```
--- stderr: astra_camera
```

```
CMake Error at CMakeLists.txt:19 (find_package):
```

By not providing "Findastra_camera_msgs.cmake" in CMAKE_MODULE_PATH this

project has asked CMake to find a package configuration file provided by "astra_camera_msgs", but CMake did not find one.

Could not find a package configuration file provided by "astra_camera_msgs" with any of the following names:

astra_camera_msgsConfig.cmake
astra_camera_msgs-config.cmake

Add the installation prefix of "astra_camera_msgs" to CMAKE_PREFIX_PATH or set "astra_camera_msgs_DIR" to a directory containing one of the above files. If "astra_camera_msgs" provides a separate development package or SDK, be sure it has been installed.

Failed <<< astra_camera [0.47s, exited with code 1]

Summary: 0 packages finished [0.72s]

1 package failed: astra_camera

1 package had stderr output: astra_camera

abdullah@abdullah-sherif:~/ros2_ws\$

المشكلة بسيطة جداً: الحزمة astra_camera تعتمد على astra_camera_msgs اللي لسه مبنية، بس ال Terminal مش "شايها" لأننا معملناش source بعد ما خلصت بناء ال messages.

عشان نريخ دماغنا من ال source كل شوية، الحل الأضمن هو بناء الحزمتين مع بعض في أمر واحد، وساعتها colcon هيضبط المسلمات بينهم أوتوماتيك.

اكتب الأمر ده ومنتحل فوراً:

Bash

`cd ~/ros2_ws`

```
colcon build --symlink-install --packages-select astra_camera_msgs  
astra_camera
```

(لاحظ أنني حطيت الاسمين جنب بعض عشان بيني الرسايل الأول ويسلمها للكاميرا في نفس الجلسة).

بمجرد ما تظهر رسالة "**Summary: 2 packages finished**"، شغل الكاميرا بالأوامر دي:

Bash

```
source install/setup.bash
```

```
ros2 launch astra_camera astra_mini.launch.xml
```

(لو ملقاش *astra_mini* جرب *astra_pro.launch.xml* أو *dabai.launch.xml*، بس غالباً *mini* هو اللي بيشتغل مع البرو).

```
source install/setup.bash
```

```
ros2 launch astra_camera astra_mini.launch.xml
```

```
file 'astra_mini.launch.xml' was not found in the share directory of package  
'astra_camera' which is at  
'/home/abdullah/ros2_ws/install/astra_camera/share/astra_camera'  
abdullah@abdullah-sh
```

مبروك! البناء (Build) تم بنجاح، والخطأ الآن بسيط جداً: اسم ملف التشغيل (**Launch File**) غير صحيح.

يبدو أن النسخة الحديثة من الحزمة (Package) غيرت أسماء الملفات أو تستخدم .py بدلاً من .xml.

الحل: لنكتشف اسم الملف الصحيح الموجود عندك.

1. عرض الملفات المتاحة

اكتب هذا الأمر في التيرمينال لترى قائمة ملفات التشغيل الموجودة بالفعل:

Bash

```
/ls ~/ros2_ws/install/astra_camera/share/astra_camera/launch
```

2. اختر الملف المناسب

غالباً ستجد ملفاً باسم `dabai.launch.py` أو `astra.launch.py`. بما أن لديك **Astra Pro**، جرب تشغيل الملف العام، فهو عادةً يكتشف الكاميرا تلقائياً.

جرب هذا الأمر (الأكثر شيوعاً):

Bash

```
ros2 launch astra_camera dabai.launch.py
```

(كلمة "Dabai" هي الاسم الكودي لسلسلة **Astra** في التعريفات الجديدة).

أو إذا وجدت ملفاً باسم `astra.launch.py`:

Bash

```
ros2 launch astra_camera astra.launch.py
```

جرب `dabai.launch.py` وقولي النتيجة! (وإذا لم يعمل، انسخ لي ناتج أمر `ls` الذي كتبناه في الخطوة الأولى).

```
ls ~/ros2_ws/install/astra_camera/share/astra_camera/launch/
astra.launch.xml      deeyea.launch.xml      multi_dabai_dcw.launch.xml
astra_pro.launch.xml  embedded_s.launch.xml
multi_dabai_pro.launch.xml
astro_pro_plus.launch.xml embedded_u3.launch.xml
multi_deeyea.launch.xml
dabai_dc1.launch.xml  gemini_e.launch.xml    multi_gemini.launch.xml
dabai_dcw.launch.xml  gemini_e_lite.launch.xml stereo_s.launch.xml
dabai_dw.launch.xml   gemini.launch.xml      stereo_s_u3.launch.xml
dabai_pro.launch.xml  list_devices.launch.xml
dabai_u3.launch.xml   multi_astra.launch.xml
abdullah@abdullah-sherif:~/ros2_ws$
```

ممتاز! القائمة أكدت أن الملف `astra_mini.launch.xml` غير موجود، ولكن البديل الأفضل والأدق لكاميرتك موجود وهو: `astra_pro.launch.xml`.

بما أنك تملك **Astra Pro**، فهذا الملف هو المصمم خصيصاً لها (لأنها تفصل بين الـ RGB والـ Depth داخلياً).

شغل هذا الأمر الآن:

Bash

```
ros2 launch astra_camera astra_pro.launch.xml
```

ماذا تتوقع بعد التشغيل؟

1. إذا ظهرت رسائل كثيرة وبقي التيرمينال "معلقاً" (لا يعطيك سطر جديد للكتابة)، فهذا يعني أن الكاميرا تعمل بنجاح! (الكاميرا الآن ترسل بيانات).

2. للرؤية، افتح تيرمينال جديد (Ctrl+Alt+T) واكتب:

Bash

```
ros2 run rviz2 rviz2
```

3. داخل Rviz:

● غير **Fixed Frame** إلى camera_link.

● اضغط **Add -> By Topic**.

● ستجد camera/depth/points/ (لنقاط ثلاثية الأبعاد) أو camera/depth/image_raw/ (للمعق).

جرب وقولي: هل اشتغلت ولا ظهر خطأ "No device found"؟

Y ده اللي ظهر

0
□ أَلِف مبروك! الكاميرا اشتغلت رسمياً.

الصورة اللي بعثها بتقول إننا نجحنا في أصعب خطوة. الشكل الأبيض اللي ظاهر في النص ده هو "سحابة النقاط"

(**Point Cloud**)، وده معناه إن مستشعر العمق (Depth Sensor) شغال وبيقيس مسافات دلوقتي وبيحولها

لشكل 3D جوه الكمبيوتر.

عشان تتأكد إن كل حاجة تمام (الألوان + العمق)، اعمل الخطوتين دول في **Rviz** عشان "تنطق" الصورة:

1. إظهار صورة الكاميرا العادية (الألوان)

حالياً أنت شايف العمق بس، عشان تشوف الصورة الألوان:

1. اضغط على زر **Add** تحت على الشمال.

2. اختر تبويب **By Topic**.

3. دور على /camera/color/image_raw واختار **Image**.

4. دوس **OK**.

● (المفروض تظهر لك شاشة صغيرة تحت على الشمال فيها صورة الفيديو الطبيعي).

2. تلوين الشكل الـ 3D (دمج العمق مع الألوان)

الشكل الأبيض اللي في النص ده ممكن نخليه ياخذ ألوان الحقيقة:

1. من القائمة اللي على الشمال، اضغط على **PointCloud2**.

2. هتلاقي اختيار اسمه **Color Transformer** (حالياً هتلاقيه FlatColor أو Intensity).

3. غيره وخليه **RGB8**.

● (لو الكاميرا قرأت الألوان، النقط البيضاء دي هتتلون بنفس ألوان الواقع).

تحليل الرسائل في التيرمينال (عشان تطمن)

في الصورة، التيرمينال كاتب:

● Start UVC camera: معناه إنه شغل كاميرا الألوان.

● device started: معناه إن الكاميرا قامت بنجاح.

● Stereo is NOT SUPPORTED: تجاهل الرسالة دي، دي طبيعية جداً لأن Astra Pro مش كاميرا "ستيريو"

(بعدستين زي العين)، هي بتستخدم تقنية تانية (Structured Light)، فالرسالة دي مجرد تنبيه ومش خطأ.

جرب تظهر الألوان وقولي، هل ظهرت صورتك ولا لسه شاشة سوداء؟ (عشان لو سوداء هحتاج خطوة صغيرة لتفعيل الـ

UVC).

Y: ده الي ظهر ايه

ألف مليون مبروك! إحنا كده وصلنا للنجاح! الكامل.

الصورة دي هي "شهادة التخرج" من مرحلة المشاكل. اللي ظاهر قدامك ده معناه إن الكاميرا شغالة بكل S_d طاقتها (صوت وصورة وعمق) على نظام ROS 2 مباشرة بدون أي محاكيات.

عشان تكون فاهم أنت شايف إيه في الشاشة (Rviz):

1. الشاشة الصغيرة تحت (أنت ظاهر فيها): دي كاميرا RGB (الألوان) شغالة زي الفل.

2. الشكل الملون في النص (3D): دي Point Cloud (سحابة النقاط). الألوان اللي فيها (قوس قزح) دي بتعبر عن "البعد والقرب" (اللون الأحمر قريب، الأزرق بعيد). دي اللي الروبوت بيستخدمها عشان يعرف إن "هنا فيه حيطه" أو "هنا فيه عائق".

3. الرسائل في التيرمينال (Dropping message): متقلقش منها خالص، دي طبيعية جداً لأن Rviz بيحاول يرسم كمية بيانات ضخمة، فاللابتوب بيقولك "أنا فوت فريم واحد عشان ألحق أرسم اللي بعده"، وده مبيأثرش على شغل الروبوت.

إيه الخطوة الجاية يا هندسة؟ (بما إن "عيون" الروبوت فتحت)

دلوقتي الكاميرا جاهزة، تحب نوظفها في إيه لمشروعك؟ قدامنا طريقتين ممتعين جداً:

الخيار 1: بناء خريطة للمكان (SLAM) نستخدم الكاميرا عشان الروبوت يتمشى في الغرفة ويرسم خريطة 3D للمكان (زي المكتبة الذكية بس أذكى بكثير). هنستخدم حزمة اسمها rtabmap_ros.

الخيار 2: الذكاء الاصطناعي وكشف الأشياء (YOLO / AI) □ نخلي الروبوت "يفهم" اللي شايفه (يقولك: دي زجاجة، ده إنسان، دي طرايزة) ويميز بينهم.

اختار واحد فيهم عشان نطبقه سواء، وأنا معاك!

Y طيب عايز منك ملف توضح فيه خطوات التشغيل بالاموامر الخاصه بالترمنال بشرحها وكمان المشاكل اللي وجهتنا وحلها
o
u
تفضل، قمت بتجهيز "دليل مرجعي شامل" لك. هذا الملف يحتوي على خطوات التشغيل السريع للمرات القادمة (وهو
s
الأهم لك)، ثم توثيق لعملية التثبيت الكاملة والمشاكل التي واجهناها وحلناها لتكون مرجعاً لك في المستقبل.
a
i
d
يمكنك نسخ هذا المحتوى وحفظه في ملف نصي (Text File) أو Notion باسم Astra_Pro_Guide.md.

□ دليل تشغيل كاميرا Orbbec Astra Pro (ROS 2 Humble)

□ أولاً: خطوات التشغيل اليومية (كل مرة تشغل فيها الروبوت)

هذه هي الخطوات التي ستقوم بها في كل مرة تريد استخدام الكاميرا مستقبلاً.

1. توصيل الكاميرا

● وصل كابل الـ USB الخاص بالكاميرا باللابتوب (تأكد من استخدام منفذ USB 3.0 للأداء الأفضل، لكن 2.0 يعمل أيضاً).

2. تشغيل الكاميرا (Terminal 1)

افتح تيرمينال جديد واكتب الأوامر التالية:

Bash

```
# تفعيل مساحة العمل (ضروري في كل تيرمينال جديد)
```

```
source ~/ros2_ws/install/setup.bash
```

```
# تشغيل الكاميرا باستخدام ملف الإطلاق الخاص بنسخة الـ Pro
```

```
ros2 launch astra_camera astra_pro.launch.xml
```

(إذا لم تظهر أخطاء وتوقف الكلام عند *device started*، فالكاميرا تعمل).

3. عرض الصورة والبيانات (Terminal 2)

افتح تيرمينال جديد لرؤية ما تراه الكاميرا:

Bash

```
# تفعيل الـ ROS
```

```
source /opt/ros/humble/setup.bash
```

```
# تشغيل أداة العرض
```

```
ros2 run rviz2 rviz2
```

داخل **Rviz**:

1. غيّر **Fixed Frame** إلى camera_link.

2. اضغط **Add -> By Topic**.

3. اختر camera/color/image_raw/ (للصورة الألوان).

4. اختر camera/depth/points/ (للرؤية ثلاثية الأبعاد Point Cloud).

ثانياً: توثيق عملية التثبيت (للمرجع فقط)

هذه الخطوات قمنا بها بالفعل ("One-time setup")، ولا تحتاج لتكرارها إلا إذا حذفت النظام أو قمت بتثبيته على جهاز جديد.

1. تثبيت المكتبات الأساسية

Bash

```
sudo apt update
```

```
sudo apt install libuvc-dev libgflags-dev nlohmann-json3-dev libgoogle-glog-dev
```

```
ros-humble-image-transport ros-humble-image-publisher
```

2. تجهيز مساحة العمل (Workspace)

Bash

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws/src
```

```
git clone https://github.com/orbbec/ros2_astra_camera.git
```

3. تثبيت قواعد ال (USB (Udev Rules

للسماح للنظام برؤية الكاميرا دون الحاجة لصلاحيات sudo في كل مرة.

Bash

```
cd ~/ros2_ws/src/ros2_astra_camera/astra_camera/scripts
```

```
sudo bash install.sh
```

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

4. بناء الحزمة (Build)

Bash

```
cd ~/ros2_ws
```

```
colcon build --symlink-install --packages-select astra_camera_msgs
```

```
astra_camera
```

⚠️ ثالثاً: سجل المشاكل والحلول (Troubleshooting Log)

هذه هي المشاكل التي واجهناها أثناء التثبيت وكيف تم حلها، لتوثيق الخبرة الهندسية.

المشكلة 1: خطأ في ربط بايثون (Python Linking Error)

● **الخطأ:** ظهور رسالة recompile with -fPIC وفشل البناء بسبب /usr/local/lib/libpython3.10.a.

● **السبب:** وجود نسخة بايثون مثبتة يدوياً (Static Library) تتعارض مع نسخة النظام.

● **الحل:** إخفاء الملف المسبب للمشكلة لإجبار النظام على استخدام النسخة الأصلية.

Bash

```
sudo mv /usr/local/lib/libpython3.10.a /usr/local/lib/libpython3.10.a.bak
```

المشكلة 2: مكتبة libuvc مفقودة

●الخطأ: None of the required 'libuvc' found.

●السبب: الحزمة تحتاج لمكتبة التعامل مع الـ USB Video Class وهي غير مثبتة افتراضياً.

●الحل: تثبيت المكتبة الناقصة.

Bash

```
sudo apt install libuvc-dev
```

المشكلة 3: اختفاء ملف astra_mini.launch.xml

●الخطأ: file 'astra_mini.launch.xml' was not found.

●السبب: تحديث الحزمة من المصدر وتغيير أسماء الملفات، بالإضافة لامتلاكنا نسخة "Pro" التي تتطلب ملفاً مختلفاً.

●الحل: استخدام ملف الإطلاق الصحيح المتوافق مع الكاميرا:

Bash

```
ros2 launch astra_camera astra_pro.launch.xml
```

نصيحة أخيرة: احتفظ بهذا الملف في مجلد المشروع، سيوفر عليك ساعات من البحث في المستقبل! بالتوفيق يا

هندسة في مشروع الروبوت. ☐

Y طيب عايز امسح الويندوز الل ينزلناه والفرشوال بتاعه اللي ادناه 50 جيجا ده امسح كل حاجه متعلقه بيه بحيث اوفر مساحة
الاب مبروك نجاح تشغيل الكاميرا الجديدة! دلوقتي نقدر نمسح "الهم القديم" (VirtualBox والويندوز الوهمي) ونسترجع
الـ 50 جيجا اللي أخذناها من الهارد.

s
a
i
d

إليك خطوات الحذف الكامل والتنظيف عشان ترجع مساحتك:

1. حذف الماكينة الوهمية (Windows VM)

دي الخطوة الأهم لأنها هي اللي واكله الـ 50 جيجا.

1. افتح برنامج **VirtualBox**.

2. في القائمة اللي على الشمال، هتلاقي اسم الويندوز اللي عملناه (مثلاً Windows 10 أو KinectWin).

3. اضغط عليه كليك يمين واختار **Remove**.

4. هيطلعلك رسالة تحذيرية، اختار منها: **Delete all files** (مهم جداً تختار ده مش **Remove only**. عشان

يمسح الهارد الوهمي من الجهاز ويرجع المساحة).

2. حذف برنامج **VirtualBox** نفسه (اختياري)

لو مش ناوي تستخدم أنظمة وهمية تاني وعازب تنظف الأوبونتو تماماً:

افتح التيرمينال (Ctrl+Alt+T) واكتب الأوامر دي:

أولاً: إزالة البرنامج الرئيسي

Bash

```
sudo apt remove --purge virtualbox virtualbox-qt virtualbox-dkms
```

(كلمة - - purge بتمسح البرنامج بإعداداته).

ثانياً: تنظيف الملفات المتبقية (**Dependencies**)

Bash

```
sudo apt autoremove
```

3. تنظيف نهائي (لضمان رجوع كل المساحة)

ساعات VirtualBox ييسبب ملفات في الـ Home folder. عشان تتأكد إن كل فتقوتة اتمسحت:

1. افتح مدير الملفات (Files).

2. اضغط Ctrl+H عشان تظهر الملفات المخفية.

3. دور على مجلد اسمه .config/VirtualBox وامسحه.

4. دور على مجلد اسمه VirtualBox VMs (عادة بيكون في الـ Home مباشرة) وامسحه لو لسه موجود.

كده مبروك عليك رجوع الـ 50 جيجا كاملة، وجهازك بقى خفيف وجاهز لمشروع الروبوت!

Y دلوقتي انا شغال علي مشروع autonomous manipulator robot arm

وده فايلات توضح وصلنا لفين فيه

o
u

تمام، أنا ألقيت نظرة سريعة .على ملفات المشروع (GP1, GP2, GP3) ومعرفتي السابقة من الـ Logs اللي
ظهرد^s واحنا بنسطب الكاميرا (زي diff_drive_bridge و hub_motor_controller).

a
d

واضح إننا أمام مشروع **Mobile Manipulator** (فراع روبوت مركب على عربة متحركة)، وده مشروع قوي جداً
وتقيل هندسياً.

بناءً على الملفات دي والخطوة الناجحة اللي عملناها بتشغيل الكاميرا، دي خارطة الطريق (Roadmap) عشان نربط
الكاميرا بالمشروع ونحوّله من "روبوت بيتحرك بالريموت" لـ "روبوت ذاتي القيادة":

1. فهم دور الكاميرا في المشروع (The Perception Layer)

الكاميرا الـ Orbbec Astra Pro اللي شغلناها دلوقتي هي "عين" الروبوت. وظيفتها في المشروع هتكون حاجة من
اتنين (أو الاتنين مع بعض):

● **Visual SLAM**: استخدام الكاميرا لرسم خريطة للمكان عشان الروبوت يعرف هو فين (بدل الـ Lidar أو معاه).

● **Object Manipulation**: استخدام الكاميرا عشان الفراغ يشوف "الجسم" اللي عايز يمسه، ويحسب مكانه (

x, y, z) بالنسبة للروبوت.

2. التحدي القادم: دمج الفراع مع الكاميرا (Hand-Eye Calibration)

عشان الروبوت يمد إيدته ويمسك حاجة شافها بالكاميرا، لازم يعرف "مكان الكاميرا فين بالنسبة لإيدته". ده بنسميه **TF**

.Tree (Transform Tree)

هنا لازم أسألك سؤال مهم عشان أحدد الخطوة الجاية: هل الكاميرا مثبتة على جسم الروبوت (**Base**) وبتبص على الفراع؟ (**Eye-to-Hand**) ولا مثبتة على آخر نقطة في الفراع (**Gripper**) وبتتحرك معاه؟ (**Eye-in-Hand**)

3. الخطوات التنفيذية (Next Action Plan)

بما إننا شغالين **ROS 2 Humble**، دي الخطوات اللي محتاجين نعملها بالترتيب عشان المشروع يكمل:

(أ) الـ **URDF** (التوأم الرقمي) - أهم خطوة حالياً

هل عندك ملف **URDF** كامل للروبوت؟ (ملف يوصف شكل الفراع، أطوال الوصلات، ومكان الكاميرا).

● **لو موجود**: لازم نضيف عليه "Link" جديد بيمثل الكاميرا الـ **Astra Pro** في مكانها الحقيقي.

● **لو مش موجود**: لازم نبدأ نصممه فراً، لأن **ROS 2** مش هيعرف يحرك الفراع من غير ما يعرف مقاساته.

(ب) **Movelt 2** (عقل الحركة)

عشان الفراع يتحرك لوحده ناحية الهدف اللي الكاميرا شافته، هنستخدم **Movelt 2**.

● هنعمل **Movelt Configuration Package**.

● هندخل فيها ملف الـ **URDF**.

● هنعرف الكاميرا الـ **Astra Pro** كـ "3D Sensor" جوا **Movelt** عشان لما الفراع يبجي يتحرك، يتفادى العوائق

اللي الكاميرا شايفها (**Octomap**).

(ج) ربط الرؤية بالحركة (**Perception-Action Loop**)

ده السيناريو النهائي اللي هنوصله:

1. الكاميرا لقطت "علبة كنز".

2. كود الذكاء الاصطناعي (YOLO مثلاً) عرف إن دي علبة وحسب مكانها ($x=0.5m, y=0.1m, z=0.2m$).

3. نبعت الإحداثيات دي لـ MoveIt.

4. MoveIt يحسب "زوايا المواتير" المطلوبة (Inverse Kinematics).

5. الروبوت ينفذ الحركة ويمسك العلبة.

ملخص ومطلوب منك الآن:

أنت قطعت شوط كبير بتشغيل الكاميرا. عشان أقدر أساعدك في دمجها مع الكود القديم (اللي في الملفات)، محتاج أعرف:

1. نوع الميكروكنترولر: هل بتستخدم ESP32 ولا Arduino Mega ولا STM32 للتحكم في المواتير؟ وهل بتستخدم micro-ROS ولا rosserial؟

2. الـ URDF: هل عندك ملف وصف الروبوت جاهز ولا لسه؟

لو جاهز، قولي "الـ URDF جاهز" عشان نبدأ نركب عليه الكاميرا في السيموليشن الأول!

Y بص انا بالنسبة للكاميرا فهنستخدمها للاتي

احنا مفروض هنخلي الروبوت يفتح اكره بابا اللي هي lever فبالتالي هحدد مكان الاوكره عن طريق الكاميرا ويبدا

الاروبوت ارم ياخذ اكشن علشان يفتح وكذلك العربيه تتحرر اثناء فتحها وبعها يعدي من الباب عادي

ثاني حاجه محتاجين الروبوت يخس اسانسير فبالتالي حتاجينه يتعرف علي اقرار الاسانسير بحيث يختار زرار الدور الاول

ولا الثاني وهكذا وكمان هيخش الاسانسير وهيطلع احنا هنعمل map عن طريق lidar مركبينه

والكامب = هتركب بجوانب الارم علي جسم اعريه بحيث تكون ثابتة

وبنستخدم esp32s3 and rosserial

عندي urdf بالطبع

ممتاز، هذا الكلام يضعنا على أرض صلبة جداً. المشروع أصبح واضح المعالم (Door Opening + Elevator +)

(Mobile Manipulation).

بما أنك تمتلك ملف الـ **URDF**، وتستخدم **ESP32-S3** (شريحة قوية جداً)، ولديك سيناريو محدد للكاميرا (Eye-to-Hand على جسم العربة)، فهذه هي خطة العمل الفنية لتنفيذ هذه المهام المعقدة:

1. تصحيح تقني هام (Micro-ROS vs Rosserial)

بما أنك تعمل على **ROS 2 Humble**، تقنية **roscpp** أصبحت قديمة وتستخدم غالباً مع **ROS 1**. البديل الرسمي والأقوى لـ **ROS 2** على الـ **ESP32** هو **micro-ROS**.

● لماذا؟ الـ **ESP32-S3** ثنائي النواة وقوي بما يكفي لتشغيل **micro-ROS Agent**، مما يجعله "Node" حقيقي داخل شبكة **ROS 2**، وليس مجرد مستقبل للأوامر. هذا سيعطيك سرعة استجابة (Real-time) أفضل بكثير لحركة الفراغ الدقيقة.

2. التحدي الأول: فتح الباب (Mobile Manipulation)

هذه أصعب مهمة في المشروع لأنها تتطلب توازن بين "حركة الفراغ" و"حركة العجل".

سيناريو الحل المقترح:

1. الرؤية (**Vision**): الكاميرا (الثابتة على العربة) ستلتقط صورة الباب. باستخدام موديل ذكاء اصطناعي (مثل **YOLOv8**)، نحدد مكان "الأكزة" (**Bounding Box**).

2. العمق (**Depth**): نأخذ النقطة المركزية للأكزة ونحسب بعدها (x, y, z) بالنسبة للكاميرا.

3. التحويل (**TF Transform**): هنا يأتي دور الـ **URDF**. يجب تحويل إحداثيات الأكزة من **camera_link** إلى **base_link** ثم إلى **arm_base_link**.

4. التنفيذ (**Action**):

● الروبوت يقترب حتى تصبح الأكزة في مجال عمل الفراغ.

● الفراغ يمسك الأكزة (**Movel**).

● الحركة الصعبة: الفراغ يقوم بإزالة الأكزة، وفي نفس اللحظة العربة ترجع للخلف (**Backwards**) والفراغ يمتد للأمام (**Extend**) للحفاظ على المسكة. هذا يتطلب كود يبعث **cmd_vel** للعجل و **joint_trajectory** للفراغ في نفس الوقت.

3. التحدي الثاني: الأسانسير (Elevator Buttons)

هنا الدقة هي الأهم. الزرار مساحته صغيرة جداً.

سيناريو الحل:

1. الاقتراب: الروبوت يستخدم Lidar للدخول والوقوف أمام اللوحة.

2. الكشف (**Detection**): الكاميرا تتعرف على الأزرار (يمكن استخدام OCR لقراءة الأرقام 1، 2، G).

3. الضغط: الفراغ يتحرك لإحداثيات الزرار، لكن بحذر (Force Feedback لو متاح، أو بحركة بطيئة) حتى لا يكسر اللوحة أو يضغط بقوة زائدة.

4. الخطوة التنفيذية القادمة (تعديل الـ URDF)

بما أن الكاميرا "ثابتة" بجوار الفراغ، يجب أن نضيفها في ملف الـ URDF فوراً لكي يفهم الروبوت مكان "عينه".

أطلب منك الآن: افتح ملف الـ URDF الخاص بالروبوت، وأضف هذا الجزء (Joint & Link) لتمثيل الكاميرا. هذا كود مبدئي يمكنك التعديل عليه:

XML

```
<"link name="camera_link">
<visual>
<geometry>
<box size="0.04 0.165 0.03"/> </geometry>
</"material name="black>
<visual/>
<link/>

<"joint name="base_to_camera" type="fixed">
</"parent link="base_link"/> <child link="camera_link">
</"origin xyz="0.2 0.0 0.15" rpy="0 0 0>
```

```
</joint/>
```

```
<"joint name="camera_optical_joint" type="fixed">
```

```
</"parent link="camera_link>
```

```
</"child link="camera_depth_optical_frame>
```

```
<origin xyz="0 0 0" rpy="-1.57 0 -1.57"/> </joint>
```

```
</"link name="camera_depth_optical_frame>
```

المطلوب منك:

1. قس المسافة الحقيقية بالمسطرة بين "سنتر العربة" وبين "مكان تثبيت الكاميرا" وعدّل قيم xyz في الكود السابق.

2. ضع هذا الكود في ملف الـ URDF.

3. شغل الـ robot_state_publisher وافتح Rviz لترى هل الكاميرا ظهرت في مكانها الصحيح أم لا.

هل قمت بقياس مكان الكاميرا بالنسبة للفراغ؟ هذا الرقم (x,y,z offset) هو أهم رقم في نجاح المشروع؛ لو غلط،

الفراغ "ميخيش" في الهواء ومش هيمسك الأكرة.

Y طيب هل الكاميرا محتاجه calibration اوي اي اعدا مسبق قبل استخدامها؟

الإجابة القصيرة: نعم، وبشدة. خصوصاً لأن مشروعك يتطلب دقة عالية (مثل ضغط زر أسانسير صغير أو مسك أكرة باب).

بدون Calibration (معايرة)، الروبوت "هيحول" (Squint). يعني الكاميرا هتقول له "الزرار في الإحداثي X"، وهو

يروح هناك يلاقى نفسه ضغط في الهواء أو خبط اللوحة.

إليك أنواع المعايرة التي تحتاجها وترتيب أهميتها لمشروعك:

1. معايرة العدسة (Intrinsic Calibration) – "مهمة جداً للأسانسير"

أي كاميرا في الدنيا عدستها بتعمل انحناء بسيط في الصورة (Distortion)، زي عين السمكة كده بس بنسبة قليلة.

● **المشكلة:** الانحناء ده بيخلي الخط المستقيم يبان مقوس. لو الزرار على طرف الصورة، مكانه الحقيقي هيتخلف عن مكانه في الصورة بمليمترات. في الأسانسير، المليمتر بيفرق.

● **الحل:** بنستخدم "لوحة شطرنج" (Checkerboard) وبنصورها من زوايا مختلفة. ROS بيحسب معامل الانحناء وبيصلح الصورة أوتوماتيك.

2. معايرة المكان (Extrinsic / Hand-Eye Calibration) – "حياة أو موت للمشروع"

دي الأهم على الإطلاق. أنت قولت إن عندك URDF، وده ممتاز، بس هل ال URDF دقيق 100% زي الواقع؟

● **المشكلة:** لو ال URDF بيقول إن الكاميرا بعيدة عن الفراغ 20 سم، وفي الحقيقة هي متثبتة بمسامير ومطبوعة 3D فبقت بعيدة 20.5 سم... ال 5 مليمتر دول هيخلو الفراغ يفشل في مسك الأكرة كل مرة.

● **الحل:**

● **الطريقة اليدوية (مبدئياً):** قيس المسافات (x, y, z) بالمسطرة (البوكليس – Caliper) بدقة شديدة جداً وعدل ال URDF.

● **الطريقة البرمجية (للمحترفين):** بنستخدم easy_handeye package في ROS. بتخلي الروبوت يمسك "علامة" (Tag) ويتحرك قدام الكاميرا، وهو بيحسب المكان بدقة الميكرون.

3. معايرة المحاذاة (Depth-Color Registration)

ال Astra Pro فيها كاميرتين جنب بعض (واحدة ألوان وواحدة عمق).

● **المشكلة:** الكاميرتين بينهم مسافة (حوالي 2-3 سم). لو معملناش محاذاة، اللون مش هينطبق على العمق. يعني ممكن تشوف "لون الزرار الأحمر" بس عمقه بيجي من "الخلفية اللي وراه".

● **الحل:** دي غالباً بتيجي مطبوعة من المصنع (Factory Calibrated)، وسوفتوير Orbbec بيضبطها لوحده (Hardware Registration). بس لازم تتأكد إنك مفعّل خيار depth_registration: true في ملف ال launch.

خطة العمل (عشان متضيعش وقت):

بما إننا في مرحلة "التجربة"، مش لازم تعمل كل ده دلوقتي. ابدأ بالتدريج:

1. المرحلة 1 (التجربة السريعة):

●اعتمد على معايرة المصنع.

●قيس مكان الكاميرا بالنسبة للفرع بالمسطرة بدقة وعدل ال URDF.

●جرب تخلي الروبوت يروح يلمس "نقطة" أنت عارف مكانها. لو لمستها صح، كمل شغل.

2.المرحلة 2 (لو الدقة مش كويسة):

●لو لقيت الروبوت بيخطيء بمسافة ثابتة (دايماً بيضرب يمين الزرار بـ 1 سم)، يبقى المشكلة في قياس ال (

Extrinsic) – < عدل ال URDF.

●لو بيخطيء في الأطراف بس وبيصيب في النص، يبقى محتاج معايرة عدسة (Intrinsic).

نصيحة: اطبع ورقة "شطرنج" (Checkerboard 8x6) والزقها على كرتونة دلوقتي، هتحتاجها قريباً جداً لما تلاقي الفرع مش راضي يظبط على الكرة.